



Implementation Strategies for Views over Property Graphs

SOONBO HAN, University of Pennsylvania, USA

ZACHARY G. IVES, University of Pennsylvania, USA

The need to query complex interactions and relationships has motivated interest in *property graph* database platforms. For some graph applications, *graph views* are required to abstract the data, e.g., to capture individual-level vs. organization-level relationships; or show single computational steps vs. composite workflows. Emerging efforts to standardize graph query languages have developed semantics and language constructs for graph views.

This paper considers the task of *implementing* such views using rewriting techniques — both using existing property graph DBMSs and converting to relational RDBMSs. We consider both virtual and materialized views, ways of rewriting queries, and structures for indexing data. We also note a common use case of graph views, which involves preserving a graph except minor local transformations; we develop novel extensions and semantics for this. We evaluate and compare the performance of our techniques under a variety of workloads, and we compare existing graph and relational DBMS platforms.

CCS Concepts: • **Information systems** → **Data management systems**; **Graph-based database models**; **Database views**.

Additional Key Words and Phrases: graph databases, storage, indexing, views

ACM Reference Format:

Soonbo Han and Zachary G. Ives. 2024. Implementation Strategies for Views over Property Graphs. *Proc. ACM Manag. Data* 2, 3 (SIGMOD), Article 146 (June 2024), 26 pages. <https://doi.org/10.1145/3654949>

1 INTRODUCTION

Property-graph databases [8, 9, 15] have gained significant momentum in handling data with complex relationships. The property graph data model combines aspects of relations (entities and relationships are represented by nodes and edges with named properties) with those of graph data models (the ability to match complex subgraph patterns via query templates), potentially even incorporating RDF triples and JSON trees. As a result, a variety of applications, including social networks, the semantic web, provenance graphs, and knowledge graphs are using graph DBMS technology. Many of these applications involve data with significant structural complexity and multiple levels of abstraction. For example, the data provenance is encoded as a PROV graph relating entities, actions, and agents [43], which might be desirable to view at different levels of detail. Knowledge graphs capture concept-subconcept and concept-instance relationships, in which transitive relationships might be explicitly or implicitly encoded. Social networks capture connections among individuals, but we might wish to instead look at abstracting these to organization-organization connections.

There is a compelling vision of “graph data warehouses” as a means of integrating diverse data, and allowing users to inspect or query relationships, potentially at different levels of detail. Such

Authors’ addresses: Soonbo Han, Computer and Information Science Department, University of Pennsylvania, 3330 Walnut Street, Philadelphia, PA, USA, soonbo@cis.upenn.edu; Zachary G. Ives, Computer and Information Science Department, University of Pennsylvania, 3330 Walnut Street, Philadelphia, PA, USA, zives@cis.upenn.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2024/6-ART146
<https://doi.org/10.1145/3654949>

applications suggest the need to support multiple *views* over the graph data, rather than simply relying on “raw” graph database instances. Existing systems and their query languages have not supported views to this point. However, following up on the G-CORE initiative, an ISO working group (<https://www.gqlstandards.org/>) has proposed a draft standard query language, GQL, which unifies ideas from vendor-developed languages such as Cypher [28], PGQL [49], and GSQL [23]. An important product of this effort is standardized languages to define graph schemas and views. Indeed, GQL views are expressive enough to integrate different property graph databases into a single output graph, and to do complex reasoning and aggregation.

Much of the GQL specification has been informed by prior query languages. However, there are open questions in whether there should be additional functionality based on graph use cases; and how to formalize and adapt these constructs, particularly views, to run on different DBMSs. For instance, in principle, a middleware layer could support GQL and implement graph views if we answer several questions: What are the underlying query reformulation [38] techniques that rewrite queries and respect the graph schema; How do we encode views and maintain them incrementally [30]; Which indexing strategies speed processing. Ideally, we would know how useful view rewriting techniques from current relational DBMSs are in this context, and how they compare to graph DBMSs. Evaluation over real problems might point to further development in the GQL and related specifications. In this paper, we seek to build upon well-understood techniques from the literature, in particular recursive Datalog and tuple-generating dependencies (TGDs), as a basis of doing graph query rewriting. Based on our application of views to real problems, we define an extension to the GQL view construct to capture a common usage pattern, namely substitution of subgraphs. To speed up execution in a relational setting, we also develop a novel *subgraph substitution relation* (SSR) indexing strategy that speeds querying over views, providing a key component to making RDBMS implementations competitive with dedicated property-graph DBMSs. Finally, we evaluate the strengths of modern property graph DBMSs, versus rewriting queries over relational DBMSs.

Furthermore, our experience with graph databases in the biomedical and social sciences suggests a common pattern in which a view performs a *set of local transformations* on the input graph (replacing a few subgraphs, preserving the rest of the graph). For example, in provenance tracking, one may want to “zoom out” [12–14] or “zoom in” in detail within a process — substituting a node for a subgraph, a subgraph for a node, etc. Similar patterns are observed in social networks. This type of usage is commonplace, but surprisingly inconvenient to express in proposed graph view languages.

EXAMPLE 1.1 (UNIFYING PROVENANCE REPRESENTATIONS UNDER ONE MODEL). *Within the data provenance community, it is common to encode provenance as a graph. Provenance capture and event logging systems [36, 44, 46, 48] often capture program execution and file I/O events in a low-level, log-style encoding with records describing object updates and values (e.g., timestamps). It is desirable to have a higher-level notion of provenance, which captures aggregate dataflow rather than sequences of events, and unifies data captured with different systems into a single model. One standard encoding is the W3C’s PROV [43] recommendation for provenance, which, unlike the “base” data is in triples, a special case of property graphs. To develop a single unified PROV view, we may want to define views over the other representations: e.g., for event-based provenance, we must elide sequences of reads and writes made to successive intermediate file versions, and instead present direct links between inputs, program runs, and outputs. As an example, consider the graph in Figure 1(a), where the same program reads f_{in} , makes a series of updates to temporary files f_2 and f_3 , and finally produces f_{out} . To convert to PROV, we may want to elide the steps involving temporary files, and add information about the user who initiates the activities. See Figure 1(b).*

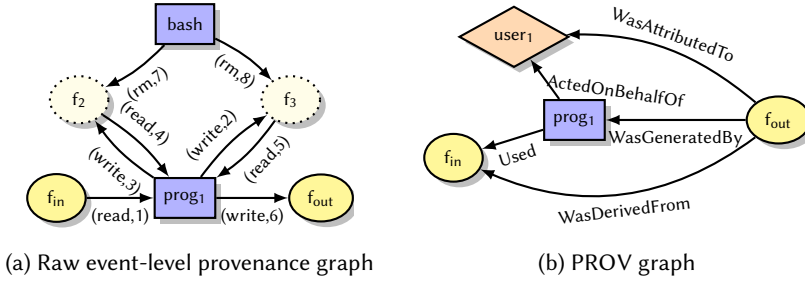


Fig. 1. An (a) event log-based provenance graph encodes I/O (`prog1` reads and writes files $f_{in}, f_2, f_3, f_{out}$); when (b) converting to PROV, we omit the temporary files, add the user, and add a variety of transitive edges.

Similar notions of *eliding* and *zooming out* on content apply equally to unified views of social networks (where, e.g., we want to convert different kinds of re-tweeting, liking, and responding to posts as a single "follows" relationship between posting user and responding user, or where we might want to summarize user-to-user interactions in more compact group-to-group interactions); to knowledge graph hierarchies (where we might want to annotate entities with the labels of *all* classes to which they belong); to UML and other diagrams (where we might want to collapse small collections of functions into broader blocks).

As noted above, the emerging GQL standardization effort supports views that bind to patterns over the input graph and selectively copy or construct nodes and edges into a target graph. Unfortunately, for the above example, the view should preserve the majority of the base graph *except for specific subgraph substitutions*. With path patterns in GQL, it can be extremely cumbersome to copy all nodes and edges, except for the ones to be elided. (We shall show details in Section 3). In fact, the above example is representative of a class of applications where collapsing or "zooming" has been proposed as a basic query operation [12] to hide proprietary details. These zooming steps can be applied to social or knowledge graphs, as well as provenance ones, either for visualization or reasoning. In this paper, we propose an additional (optional) construct for views, namely a *default rule* that copies all nodes and edges of an input graph *except* those that have been explicitly mapped or transformed. We also study how to implement both "traditional" views as proposed in GQL, as well as our extensions.

Our contributions are as follows:

- A means of converting graph views in GQL into relational formalisms, for which we leverage existing techniques for computing views and composing queries over them.
- Extensions to GQL to support the common case of *graph transformation* with subgraph substitution; and a notion of *well-behavedness* that preserves clean semantics.
- Strategies for materializing and updating views in property graph DBMSs; and for answering queries over both materialized and virtual views using relational DBMSs.
- Indexing techniques to speed the process of answering queries over graph views in relational DBMSs.
- Experimental results showing that our methods perform well, across a variety of graph transformation workloads, and on several different DBMS platforms.

2 PRELIMINARIES

Graph databases are particularly convenient for reasoning about heterogeneous entities with complex and indirect relationships, via path expressions and subgraph matching. However, as we illustrated in the previous section, at times there may be different representations of the graph

we wish to query: raw logged provenance events vs higher-level depictions of dependencies; relationships among individuals vs relationships among organizations; detailed workflows specified as subgraphs, versus “collapsed” representations that elide details.

The effort to standardize graph views [8, 21] and schemas [8, 9] is still ongoing within the Linked Data Benchmark Council and ISO working groups, but we incorporate many of the core ideas and language in our implementation. In order to develop an implementation, we map from (the core of) GQL and PG-Schema to an underlying set of relational constructs and constraints — for which an array of existing techniques from the query rewriting literature can be applied. In this section, we sketch the basics of views and constraints in GQL and PG-Schema, as well as how we represent these.

2.1 GQL/G-CORE Overview

Views in GQL (and the G-CORE proposal [8] that forms much of its foundation) build on *conjunctive path patterns* over graphs, in the form of the Graph Pattern Matching Language (GPML) [21]. Influenced by the Cypher language [28], GPML specifies path expressions using an “ASCII art” method in which nodes and edges are specified within parentheses and square brackets, respectively, and the direction of edges is indicated by “arrows”. For example, a pattern from Figure 1 might take the form $(\text{fin}:\text{Entity})-[:\text{read}]\rightarrow(\text{prog1}:\text{Activity})$. Within parentheses or brackets, we use a *variable name:label* syntax.

GQL uses a CONSTRUCT *nodes-and-edges* WHERE *predicates* MATCH *GPML-patterns* syntax — analogous to the SQL clause structure. Corresponding to SQL SELECT, the CONSTRUCT clause can *output into a new graph* nodes, edges, and paths copied from bound variables from the rest of the query. Essentially, the graph nodes and edges in the CONSTRUCT clause are copied in an identity-preserving way to the graph; and as specified new nodes and edges are constructed through computed expressions. The MATCH clause takes the place of SQL’s FROM, but allows for specification of regular path expressions using GPML path-patterns. The WHERE clause, as in other languages, allows predicates over bound variables and over paths. We illustrate more via simple examples below.

EXAMPLE 2.1. *Revisiting our provenance integration scenario, a view definition in GQL, posed over the schema of Figure 1(a), might look like this:*

```
CREATE VIEW view AS (
  CONSTRUCT (f1)<-[u:Used]-(p) SET u.ts = 3
  MATCH (f1:Entity)-[:read]->(p:Activity) )
```

The view creates a new graph that copies an Entity node, f1, from the input when f1 is connected by a read edge to an Activity bound to variable p. A new edge, u, is constructed in the view, and its ‘ts’ (timestamp) property is set to a literal value. The WHERE clause is omitted.

Each CONSTRUCT-MATCH-WHERE expression returns a set of nodes and edges; multiple graph expressions can be UNIONed together into a (de-duplicated) set of nodes and edges forming an output graph. In this paper, we follow the GPML specification [21], where each GQL CONSTRUCT-MATCH-WHERE expression in a union has only conjunctive path steps (without negation). Thus a CONSTRUCT clause represents a conjunctive path query, extended with inequality/unequality between variable bindings and literals; and a view represents the union of such queries.

2.2 Graph and Schema Model

While our goal is to implement a property graph data model for both the base graph and views, it is often natural to formalize graphs and views by going back to the notion that a graph $G = (N, E)$ is a set of nodes N (these nodes, including properties, can be captured as relations) and edges E (which also can be captured as relations linking pairs of nodes). Based on that intuition, we formalize the

core elements of the draft GQL and PG-Schema efforts [9, 10] in a relational encoding. We capture the basic components of the property graph in a relational representation, which ultimately can be mapped to actual relations (in a relational DBMS implementation) or to the components of a property graph (in a graph DBMS implementation).

2.2.1 Property Graph Relations. We model the graph's nodes and edges using relations in a schema Σ , to which we will add various constraints to establish graph schemas. For simplicity of exposition we assume a single unnamed graph; but our model naturally extends to multiple graphs.

$N(n, label)$	Labeled nodes, by ID
$E(e, n_1, n_2, label)$	Labeled edges, by ID
$NP(n, key, value)$	Node key-value properties
$EP(e, key, value)$	Edge key-value properties

where both nodes and edges may have unique IDs. Our representation is general enough to support multiple nodes with the same label as well as multi-edges between pairs of nodes.

2.2.2 Graph Schemas and Constraints. The PG-Schema [9, 10] proposal develops a set of types for nodes and edges, and establishes constraints based on these. We implement a subset of this specification: we focus on *closed types*, in which content must explicitly allowed by the type; and we limit each node or edge label to a single type. To capture types, we define several auxiliary relations:

$NT(label, nt)$	Mapping: node label to node type
$ET(label, et)$	Mapping: edge label to edge type
$NPROPS(nt, key, type)$	Labels, types for node properties
$EPROPS(et, key, type)$	Labels, types for edge properties

EXAMPLE 2.2. Consider an excerpt from a graph schema corresponding to the PROV view in Figure 1(b):

```
CREATE GRAPH TYPE provGraphType STRICT {
  (agentType: Agent {id INT, name STRING}),
  (activityType: Activity {id INT, startTime DATETIME,
    endTime DATETIME}),
  (:activityType)-[actedType: ActedOnBehalfOf]->
    (:agentType),
  FOR (x:agentType) IDENTIFIER x.id,
  FOR (x:activityType) IDENTIFIER x.id,
  ... }
```

This establishes two node types (*agentType* and *activityType*) and an edge type (*actedType*); and associates these with nodes labeled **Agent** and **Activity**, and **ActedOnBehalfOf**, respectively. It further establishes the properties allowed for each type; and adds several constraints. The **ActedOnBehalfOf** edge must have endpoints that are of types *agentType* and *activityType* (a type of foreign key); and both *agentType* and *activityType* have unique IDs (keys).

We encode the **agent** node-related constructs as tuples within our auxiliary relations for nodes: $NT("Agent", agentType)$, $NPROPS(agentType, "id", int)$, $NPROPS(agentType, "name", str)$; and similarly for **activities**. Additionally, we need to establish a set of constraints over the node and edge-related relations.

Keys. PG-Schema establishes several types of key constraints. We focus here on **IDENTIFIER** constraints, which establish that each node of a given type has a value for the "key" property (in relational terms: non-null), and that each value occurs uniquely among the set of all nodes of

this type. We can codify this using *equality-generating dependencies* (EGDs) and *tuple-generating dependencies* (TGDs) as per prior work on graph constraints [25].

EXAMPLE 2.3. *An EGD establishing uniqueness of the agent key for our provenance-graph running example would look like:*

$$\begin{aligned} \forall n_1, l_1, n_2, l_2, k_1, k_2 (N(n_1, l_1) \wedge N(n_2, l_2) \wedge NT(l_1, \text{activityType}) \wedge \\ NT(l_2, \text{activityType}) \wedge NP(n_1, \text{"id"}, k_1) \wedge NP(n_2, \text{"id"}, k_2) \\ \rightarrow k_1 = k_2) \end{aligned}$$

Additionally, we would want an inclusion dependency (a special case of a TGD) establishing the existence of the key for every activity node:

$$\forall n_1, l_1 (N(n_1, l_1) \wedge NT(l_1, \text{activityType}) \rightarrow \exists x (NP(n_1, \text{"id"}, x)))$$

Structural constraints and foreign keys. PG-Schema also establishes constraints on which edges connect which nodes: in our example above, *actedType* connects an agent to an activity. This is one of several ways in which a foreign key can be encoded in PG-Schema. We can express this using a more complex TGD:

$$\begin{aligned} \forall e, s, t, l (E(e, s, t, l) \wedge ET(l, \text{actedType}) \wedge l = \text{"ActedOnBehalfOf"}) \\ \rightarrow \exists sl, tl (N(s, sl) \wedge NT(sl, \text{activityType}) \wedge N(t, tl) \wedge NT(tl, \text{agentType})) \end{aligned}$$

2.3 Views as Tuple-Generating Dependencies

Suppose the DB administrator has defined a set of GQL views over a graph. To understand how to efficiently use this view (and answer queries over it), our goal is to map GQL to a more “standard” construct that is better understood from a database theory perspective, such that we can leverage prior (relational) techniques to reason about compositions of queries over views.

Unlike a standard relational view, a graph DBMS view outputs a combination of multiple kinds of entities (nodes, edges, properties) that are naturally represented in multiple relations. As a formalism that allows us to specify constraints over multiple relations at a time within a single expression, we adopt *tuple-generating dependencies* (TGDs) [11] to formalize each conjunctive GQL expression as a (partial) mapping from the input graph to output graph. TGDs are commonly used in data integration and exchange to map from one schema to another. A TGD over some schema Σ is of the form:

$$\forall \bar{X}, \bar{Y} (\phi(\bar{X}, \bar{Y}) \rightarrow \exists \bar{Z} (\psi(\bar{X}, \bar{Z})))$$

where ϕ (resp. ψ) is a conjunction of atoms in Σ (resp. Σ') over variables $\bar{X}, \bar{Y}$ (resp. $\bar{X}, \bar{Z}$). We will omit the universal quantifiers of TGDs for brevity. Moreover, while we will allow the creation of new values through Skolem functions, we will generate only *full TGDs* ($\bar{Z}$ is empty). As a result, applying the chase [22] to such a TGD computes a standard DBMS instance as opposed to an incomplete instance [6].

Intuitively, our system will automatically convert GQL views into TGDs. From this, applying the chase procedure over the TGD allows us to compute a conjunctive expression over a set of input relations (the graph nodes, edges, properties, etc.) and output results into a set of output relations (i.e., we can materialize the view output’s nodes, edges, properties, etc.). Multiple TGDs can be chased together, and their unioned results will form the **final contents of the graph view**. Suppose we designate the view’s output nodes and edges as the N', E', NP' , and EP' relations in the view’s relational schema Σ' .

EXAMPLE 2.4. *Continuing our running example, the view definition of Example 2.1 can be translated into the TGD:*

$$\begin{aligned}
& N(f_1, \text{"Entity"}) \wedge N(p, \text{"Activity"}) \wedge E(r, f_1, p, \text{"read"}) \\
& \rightarrow \exists u(N'(f_1, \text{"Entity"}) \wedge N'(p, \text{"Activity"}) \wedge E'(u, p, f_1, \text{"Used"}) \wedge EP'(u, \text{"prop"}, 3)).
\end{aligned} \tag{t_1}$$

As in the above example, the semantics of a construction rule can be directly expressed as a TGD.

The data integration and exchange literature [27, 29] has shown that (with the addition of *Skolem functions* to create shared IDs for any new entities or values introduced in the output) we can take a TGD and break it into a set of Datalog rules (one for each atom in the right-hand side). These rules can be used to create materialized output relations for the view, equivalent to applying the chase over the TGD(s). Conversely, queries over the output view can be unfolded with these Datalog rules to produce queries over the input graph.

The data model and constraint formalisms defined above will form the basis for how we reason about and reformulate queries and views (Section 4). Before focusing on query rewriting strategies, however, we first identify a common use case for graph views that requires novel extensions to our view language — as well as additional semantic constraints.

3 VIEWS WITH MINOR TRANSFORMATIONS

For many kinds of complex graphs, views are an important intermediate step for ensuring privacy of certain subgraph structures (e.g., “zooming out” in provenance graphs [12]), or of ensuring the content is at the right level of detail for browsing or presentation (e.g., excerpting and summarizing information within huge low-level graphs, or alternatively adding detail to sparse high-level graphs). For these kinds of *graph transformations*, the view often preserves the elements of the input graph, *except* for certain subgraphs that are specifically to be replaced (with other subgraphs, with nodes, or with nothing). In the draft GQL/G-CORE standard, a view *must explicitly* copy all elements from the input to the output, disallowing cases that match the components to be replaced — often making the definition cumbersome and redundant.

EXAMPLE 3.1. *Consider the provenance-graph transformation of Figure 1. We want a view representing the PROV graph in (b), which preserves the majority of the input graph in (a), except that it (1) removes all ellipse nodes (entities) linked to the bash activity by an rm edge representing removal of a temp file, (2) removes the bash activity (rectangle); (3) replaces read edges with Used edges in the opposite direction; and (4) replaces write operations with a WasGeneratedBy edge in the opposite direction. Additionally, a (single unique) user1 Activity (diamond) node needs to be introduced into the view, and linked to any remaining activity or written entity nodes. To accomplish this in the GQL/G-CORE draft specification, we must explicitly test each file node ($f_{in}, f_{out}, f_1, f_2$) to see if it is connected via an rm edge to a bash activity; and only copy it if not. This results in significant redundancy, which explodes combinatorially as we disallow more input patterns:*

```

GRAPH VIEW prov AS (
  CONSTRUCT (f1)<-[u:Used]-(p)
  MATCH (f1:Entity)-[:read]->(p:Activity)
  [OPTIONAL (f1)<-[:rm]-(a2:Activity)
    WHERE a2.name='bash'] ON EventGraph
  where A2 IS NULL
  UNION
  CONSTRUCT (p)<-[g:WasGeneratedBy]-(f1)
  MATCH (p:Activity)-[:write]->(f1:Entity)
  [OPTIONAL (f1)<-[:rm]-(a2:Activity)
    WHERE a2.name='bash'] ON EventGraph
  where A2 IS NULL
  UNION

```

```

CONSTRUCT (f1)<-[:WasAttributedTo]-[u:Agent]
          SET u.id=1, u.name='User1'
MATCH (p:Activity)-[:write]->(f1:Entity)
[OPTIONAL (f1)<-[:rm]-(a2:Activity)
  WHERE a2.name='bash'] ON EventGraph
  where A2 IS NULL
UNION
CONSTRUCT (p)->[:ActedOnBehalfOf]->[u:user]
          SET u.id=1, u.name='User1' )

```

We argue that this is not intuitive, and the query is difficult to extend — even though the desired subgraph transformations can each be specified compactly! A more natural semantics would define a view first by matching subgraphs, and constructing corresponding output subgraphs; then adding a form of *negation*. Namely, we could union the outputs with (identity-preserving copies of) *all nodes and edges that were not mapped by subqueries of the view*.

To allow this, we extend our view formalism, to be computed in two steps or strata [6], as follows. In the first stratum, TGDs are chased and their results are unioned together to form the output graph. A *mapping table* is recorded for all nodes and edges that were bound by these subqueries, and were thus mapped to outputs. In a subsequent stratum, a *default rule* is applied which copies *all nodes* that have not been mapped, and simultaneously maps *all edges* that were not already mapped (whose node endpoints were previously mapped to the output graph, either directly in the mapping table, or by the default rule). To allow for certain nodes and edges to explicitly not be mapped, we add a DELETE clause that marks them to not be copied.

EXAMPLE 3.2. *Revisiting our example, we can copy all elements of the graph except those related to deleted files as follows:*

```

GRAPH VIEW NoTemps AS (
  DELETE f1
  MATCH [(f1:Entity)-[:rm]-(a2:Activity)
    WHERE a2.name='bash'] ON EventGraph
  UNION DEFAULT_MAPPING )

```

Then we can naturally compose another view over this, transforming and adding the remaining edges.

```

GRAPH VIEW prov AS (
  CONSTRUCT (f1)<-[:Used]-(p)
  MATCH (f1:Entity)-[:read]->(p:Activity)
  ON NoTemps
UNION
  CONSTRUCT (p)<-[:g:WasGeneratedBy]-(f1)
    <-[:WasAttributedTo]-(u:user)
    SET u.id=1, u.name='User1'
  MATCH (p:Activity)-[:write]->(f1:Entity)
  ON NoTemps
UNION
  CONSTRUCT (p)->[:ActedOnBehalfOf]->[u:user]
    SET u.id=1, u.name='User1' )

```

The default rule contains negation, i.e., it adds all input graph nodes and edges, *except* if they have participated in a mapping in one of the components of the view. Beyond the simple DELETE with a default rule, we also allow CONSTRUCT clauses to explicitly MAP input nodes to corresponding output nodes, assigning identity based on *Skolem functions* [6] applied to the inputs. Multiple nodes and edges in an input subgraph can be mapped to a single output node or edge (as determined by

the identity of the output node/edge); multiple bound subgraphs can be mapped to the same node or edge; and new nodes or edges can be created as needed.

EXAMPLE 3.3. Consider another transformation of Figure 1. We can map or collapse subgraphs that represent intermediate activities to a new workflow activity node in the output graph. This allows us to create a simplified representation of the workflow, where intermediate steps are combined into a single node.

```
CREATE VIEW zoom AS (
  CONSTRUCT
    (f1)<-[:Used]-(w:Workflow)<-[:WasGeneratedBy]-(f3)
    MAP FROM p1, f2, p2 TO w
    SET w.id = SkolemFunc('wf', p1, p2)
  MATCH (f1:Entity)-[:read]->(p1:Activity),
    (p1)-[:write]->(f2:Entity)-[:read]->(p1),
    (p2:Activity)-[:rm]->(f2), (p1)-[:write]->(f3)
  ON EventGraph )
```

The SET clause allows us to set the identities of new nodes or edges with a Skolem function. SkolemFunc takes a Skolem function name as its first argument; then bindings from the MATCH clause as additional arguments. The function guarantees a unique ID for each combination of Skolem name and input values (and values will not collide with IDs created in any other way). Multiple elements from the input graph can be mapped to the same element on the output graph (merging or “collapsing” them); multiple rules can use the same Skolem values to create shared (merged) node or edge identities, allowing us to create complex relationships in nodes and edges mapping.

3.1 Capturing Mappings and Adding Defaults

For simplicity, we refer to views that preserve most of the input graph with a default rule as *transformation views* since they apply a small number of subgraph transformations to the input graph. Recall from the previous section that we convert each CONSTRUCT (and, in our case, DELETE) clause from a view into a TGD. To capture the mappings performed within a TGD, we add an atom to the right hand side of the TGD for each input binding-to-output node pair (n, n') or edge pair (e, e') , essentially outputting that pair into a node mapping and edge mapping table:

$N^M(n, n')$ Node to node ID mapping ($n' = \perp$ if n is deleted)

$E^M(e, e')$ Edge to edge ID mapping ($e' = \perp$ if e is deleted)

where the node and edge remappings are encoded. Every input node n or edge e in N^M or E^M is paired with a new id(s) or $\perp$. As we describe later, for semantic consistency we allow for a many-to-many association between input and output IDs, but a node or edge should not be simultaneously mapped to a new id and $\perp$.

EXAMPLE 3.4. Consider the DELETE clause in Example 3.2. This clause is essentially an ‘empty CONSTRUCT’ that returns no edges or nodes from the input graph; but maps the deleted items to the $\perp$ symbol for deletion. We would generate a TGD of the form.

$$N(f_1, \text{“Entity”}) \wedge N(a_2, \text{“Activity”}) \wedge E(e_1, f_1, a_2, \text{“rm”}) \wedge NP(a_2, \text{“name”}, \text{“bash”}) \\ \rightarrow N^M(f_1, \perp) \wedge E^M(e_1, \perp)$$

Default Rules. Any source node that is not explicitly mapped or removed by a transformation rule in a view is instead matched by a *default transformation rule* (t_3) below that copies the node

to the output, maintaining its ID. To fully capture node to node ID mappings from input graph to output graph, we also use the following relations in the default rules:

$$\begin{array}{ll} N^{DM}(n, n') & \text{Node:node ID mapping by default rules} \\ E^{DM}(e, e') & \text{Edge:edge ID mapping by default rules} \end{array}$$

We establish this through an identity mapping for all nodes not already mapped, and all such non-deleted node IDs are copied to the output:

$$N^M(n, n') \wedge n' \neq \perp \rightarrow N^{DM}(n, n') \quad (t_2)$$

$$N(n, l) \wedge \neg N^M(n, n') \rightarrow N^{DM}(n, n) \wedge N'(n, l) \quad (t_3)$$

Similarly, edges are copied by default via (t_5) below, so long as their endpoints are mapped to the output graph, they are not already mapped, and they are not explicitly marked for deletion. This will maintain referential integrity.

$$E^M(e, e') \wedge e' \neq \perp \rightarrow E^{DM}(e, e') \quad (t_4)$$

$$E(e, s, d, l) \wedge \neg E^M(e, \perp) \wedge N^{DM}(s, s') \wedge N^{DM}(d, d') \rightarrow E'(e, s', d', l) \wedge E^{DM}(e, e) \quad (t_5)$$

Node and edge properties copied by default mappings in a similar fashion; we omit details for brevity.

Putting It All Together. Given a view definition in our extended GQL language, we create a set of TGDs whose right-hand sides define the graph corresponding to the view: nodes N' , edges E' , as well as node-properties NP and edge properties EP . In addition, for transformation views, we create auxiliary relations N^M , E^M , and so on, representing the input graph elements that were mapped; then use the input graph minus those elements to produce the default mappings N^{DM} , E^{DM} , and so on. These latter derived relations include negation and must be computed in a second stratum, after the initial rules were run. Finally their output is unioned with N' , E' , and so on; forming N'' , E'' , and so on — which contain our final graph view. As we describe in detail in Section 4, we can either make these virtual views (unfolding queries over them), or we can materialize them directly. However, we first consider one additional issue: namely, that introducing MAP and DELETE operations can result in nondeterministic interactions between clauses within a transformation view, e.g., if we simultaneously delete and map a given input node.

3.2 Well-Behaved Views

Given that a graph view is computed as the union of multiple rules' transformations, and each rule may map input elements to output elements (possibly merging them) or may delete input elements — there is a natural question of when the rules are semantically inconsistent or ambiguous.

As illustrated schematically in Figure 2, a *graph transformation view* G' is the result of evaluating by a view definition (comprised of rules whose outputs are unioned together) over an input graph G (which may be the the output of another view). Let us denote the *mapping set* of transformation rules as $r_i \in \mathcal{V}$, including the default mapping rule. The set of mappings in the rule should produce *deterministic output* consistent with user expectations, regardless of execution order (or parallelism) among rule execution; and the output of the default rule, computed in a second stratum, should also be deterministic.

DEFINITION 3.1 (WELL-BEHAVED VIEW). *A graph view $\mathcal{V}$ is said to be well-behaved under graph schema Σ if it satisfies two properties.*

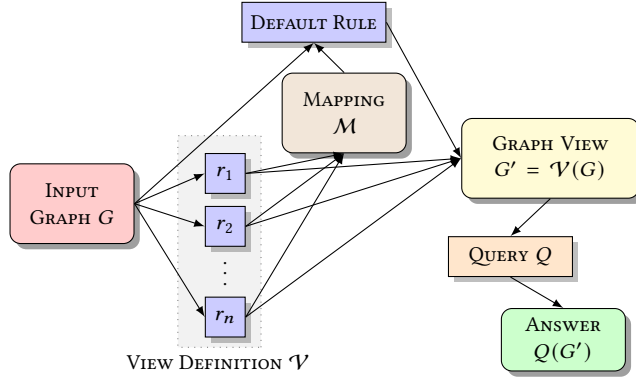


Fig. 2. An overall diagram of defining a graph transformation view by a set of transformation rules ($r_1, \dots, r_n$) and the default rule along with a mapping relation over an input graph and querying on the view.

- (1) Any mappings derived by $\mathcal{V}$ applied to a graph $G \models \Sigma$ cannot perform conflicting mappings on the same source node or edge.
- (2) The mappings do not result in an output graph that violates schema constraints, including uniqueness of labels and keys, and referential integrity.

We considered several notions of conflicting mappings, in expressing various types of transformations in provenance and other domains. The most useful definition is to preclude simultaneous *deletion* of a graph element while also *mapping* it to an output graph element. We use this definition below.

Analogously to typechecking in a programming language, it is often desirable to know when a set of transformation rules *will not interact with regards to their mapped inputs* and thus are guaranteed to not violate the first property in the previous section. We thus establish a static typechecking algorithm as a first step. We then consider cases where we may want to check at runtime if two mappings that interact may *still do so in a compatible way* and whether the output conforms to a schema.

Non-conflicting rules. A view definition consists of several transformation rules, with path patterns that can partially overlap. A key challenge, addressed in Section 3.2, is to efficiently confirm that two rules do not *conflict* with each other, i.e., that the output of one rule (perhaps a deletion) is semantically compatible with every other rule. Generally, a precisely defined schema, along with well-specified rules, should guarantee such non-conflicting rules.

3.2.1 Static Checking of Conflicting Rules. Our static compatibility checking algorithm takes a graph schema Σ and a set of transformation rules whose results are unioned together in $\mathcal{V}$ as input, and it determines whether $\mathcal{V}$ can produce conflicting mappings for the same source entities, for any graph $G \models \Sigma$.

A sketch of the algorithm is as follows. Without loss of generality, consider two rules, $\mathcal{V} = \{r_1, r_2\}$. To capture the intuition, we can define a *matching pattern graph* p_i of r_i : p_i represents the pattern “template” defined in each MATCH (and associated predicates in the WHERE) clause of r_i . We further add predicates restricting the sets of edge and node labels that can be matched by the pattern, according to the graph schema.

Next, we annotate p_i by coloring each node or edge that is not mapped to itself (i.e., not changed) by transformation. We color as red each source node in the MAP clause, and any edges connected to it. We also color as blue the node, or the edge and its endpoints, included the DELETE clause. Note

that if a node or edge in a graph G is matched by a colored node (or edge) in a pattern graph — this means it may participate as a source entity in the *mappings* relation, thus any source entity that is colored both red and blue is both remapped and deleted, i.e., incompatible.

In our actual algorithm, we model this test as an SMT (satisfiability modulo theory) problem by declaring a graph G consisting of the sets of node and edge predicates for the two subgraphs matching p_1 and p_2 , respectively; as well as the set of schema constraints. The SMT formula is designed to be *satisfiable if and only if* there exists a graph G conforming to Σ . The algorithm accepts *if and only if* the SMT formula is *unsatisfiable*, meaning there is no graph under which the colored nodes or edges overlap. Our algorithm is both sound and complete (proof omitted due to the space limit).

Our checking algorithm can be generalized to more than two rules by checking every pair of rules. Each rule needs to be checked with itself as well, because any node or edge should be remapped at most once.

Complexity. We briefly describe the complexity of the checking algorithm. Let n be the number of rules in $\mathcal{V}$. We need $\binom{n}{2} + n$ checks, each of which is NP-hard. An SMT solver finds solutions to NP-hard problems, but in practice tends to do so even at scale. In our setting, the size of the satisfiability problems is based on query complexity: namely, the number of variables (i.e., nodes and edges in the graph pattern) and that of rules (or assertions). These are small in practice, as opposed to the size of graph data. We also do the satisfiability check once for each set of transformation rules, and can memorize the results.

3.2.2 Runtime Checking of Schema Compliance. The above descriptions covered the first property for well-behavedness: namely, that the same source was not mapped in incompatible ways. In a final step, we use the DBMS' built-in capabilities to check the view output against the output schema at runtime, through a combination of key-foreign key constraints mapped from schema constraints enforced in the underlying DBMS; and queries to find violations to the EGDs and TGDs representing the schema.

4 IMPLEMENTING GRAPH VIEWS

The previous two sections of this paper gave logical formalisms for our data model, schemas, and constraints — including a formulation of GQL views as constraints. When views with default rules are used, this further adds a set of *mapping relations*, and then an additional set of TGDs (with negation over the mapping table, which is now an EDB) representing the default rule. Given these formalisms, we can now consider strategies for mapping from TGDs to executable queries and views.

Materialized graph views. Perhaps the simplest approach is through view materialization: we can materialize the graph produced by each view rule (i.e., N' , E' , etc.), by unioning together all produced nodes, edges, and their properties. Then we can directly answer queries using the resulting instance. This will typically result in fast queries, potentially at the expense of incremental maintenance costs and extra storage space. It can be implemented using today's existing graph DBMS systems, by translating from (our implemented subset of) GQL/G-CORE to existing languages such as Cypher; or to operations over relations. We describe each below.

Fully virtual graph views. A virtual view *unfolds* or composes a query over a graph view's rules (i.e., the query is executed over N , E , etc.). It does not require any additional storage beyond the base graph. The mappings to virtual output graph G' and any query over G' are directly executed on the input graph. Under a heavy update workload virtual views add no maintenance overhead; however, repeated queries may redundantly apply the transformation rule mappings.

The virtual view approach does not map cleanly to existing graph DBMS technologies, but it can be implemented using a relational engine, as we describe below.

4.1 Graph Views with Existing Graph DBMSs

We first target a property graph DBMS such as Neo4j to derive a view's output graph with Cypher. Currently available graph DBMSs do not support virtual views, so queries over the output graph have to be answered by using the materialized output graph. Our challenge lies in encoding base and derived data so we can perform view maintenance.

GQL view to Cypher materialized instance. Since we cannot use virtual views in graph DBMSs, we only use views to TGD equivalences to ensure transformations are well behaved. We rewrite views directly to Cypher to compute G' alongside G . Thus we consider two approaches to capture graph view output in Cypher:

- (1) (Update In-place) One naive approach is to use Cypher's CRUD operations and the APOC library to transform the input graph. However, this approach updates the input graph in-place, so it needs to be loaded again if we want to define other views.
- (2) (Overlay) The last approach is to have an augmented graph with additional properties on every node and edge, similar to versioning [17]. Consider a sequence of views defined over previous input graph or views. We assign a *stratum number* to each view, increased by 1 from the highest stratum of its inputs. (For a base input, the stratum number is 0.) A view at stratum number k consists of nodes and edges *created* at or before k , *deleted* after k . Views over views can be emulated by using properties *created* and *deleted*. This can keep the input graph and have multiple views as well.

Querying Cypher “materialized views.” In Neo4j, when a user query references a specific materialized output graph, we add predicates over the *gid*, *created*, and *deleted* properties for each variable appearing in the user query (as well as each intermediate step in a path). Direct translation from a GQL query to Cypher is straightforward, and allows queries to be posed over stored output graphs. However, it has some limitations:

- View maintenance is expensive. In the *update in-place* approach, updates to the base graph may not be feasible as the base graph is no longer available. In the *overlay* approach, incremental view maintenance is expensive as it consists of multiple queries to update portions of the graph.
- Query optimization is limited. In general, we cannot integrate extra optimization techniques, other than leveraging property graph indices.

Directly using virtual views is not feasible in any Cypher-based graph DBMS we are aware of: key building blocks such as query unfolding are not supported. In principle we can use Cypher to extract node, edge, and other relations, and then adapt the TGD mechanisms and standard relational query rewriting techniques over these, to compute answers. However, this does not take advantage of any of the key benefits of a graph DBMS, instead using it as a tuple engine. Thus we instead consider a relational DBMS implementation, which we will evaluate against Neo4J.

4.2 Graph Views with Relational Languages

We consider both native-Datalog and native-SQL relational database platforms. For both cases we leverage Datalog as an intermediate representation. Our transformation rules are based on conjunctive path queries. As described in Section 2.3, each rule $r \in \mathcal{V}$ corresponds to a tuple-generating dependency. In turn, each TGD can be mapped to a set of Datalog rules; and the Datalog program defines the target view instance following the approach outlined in [29], corresponding to executing the *chase* [24] over the TGDs (which are all full TGDs with Skolems). Not only does

this allow us to define a clean semantics for computing the instance, but query containment and rewriting are well understood with TGDs [18]. From the TGD-to-Datalog translation, we have a means of specifying N' , E' , NP' , EP' as views.

Direct execution in Datalog DBMS (e.g., LogicBlox). For each TGD r of the form $\phi(\bar{X}, \bar{Y}) \wedge \text{Skolems}(\bar{Z}) \rightarrow \psi(\bar{X}, \bar{Z})$ in our view definition, we define a separate Datalog rule:

$$\psi_i(\bar{X}_i, \bar{Z}'_i) :- \phi(\bar{X}, \bar{Y}) \wedge \text{Skolems}(\bar{Z})$$

where $\psi_i(\bar{X}_i, \bar{Z}'_i)$ is the i th atom. In this fashion, the set of TGDs derived from a view definition can be easily converted to a set of Datalog rules that define a set of IDBs comprising the nodes, edges, their properties, deltas, and mapping relation, as defined in Section 2.

Datalog to SQL. We leverage standard techniques to convert each Datalog rule to a corresponding SQL statement; and to define SQL views as the union of all translated rules with the same atom name. Skolem functions are implemented by user defined functions. The updated graph is captured by the set of views for nodes, edges, and their properties.

4.3 Answering Queries over Views

The previous section established how to create a Datalog program to create our graph view. As in the relational DBMS world, we would like to be able to evaluate queries not only over materialized view instances, but also to be able to treat the view as *virtual* and to use *view unfolding* to answer our queries.

Querying Datalog Program. Each MATCH clause is directly rewritten to the body of a conjunctive Datalog rule, except that we make use of the N' , E' , NP' , and EP' relations as the components of the graph (instead of N , E , etc.). We further add conjunctive constraints based on the WHERE clause. Subsequently, we take the various components of the CONSTRUCT clause and generate separate Datalog rules to output a new set of edges (relation E_o), nodes (N_o), edge properties (EP_o), and node properties (NP_o) that form a graph as answer.

Querying SQL views. We can directly answer the SQL query, translated from the Datalog program above using standard techniques, over the defined SQL views using the underlying DBMS. (Note that in-DBMS SQL view unfolding is significantly more efficient than first directly unfolding the Datalog rules and then running the resulting SQL queries, in part because the latter approach suffers from combinatorial explosion in the number of rewritings.)

5 ACCELERATING QUERIES OVER VIEWS

Intuitively, if we want to speed up user queries over the graph (i.e., N' , E' , etc.), we would like to materialize the output view; but the output view instance may be expensive to maintain under updates. We consider several different storage strategies for graph views, each of which offers advantages under different query and update mixes. Each approach materializes specific relations that can be used to speed up queries over the views. All storage strategies described in this section assume incremental maintenance is required when the input graph is updated. We compare the performance of storage strategies in Section 6.

Between the extremes of virtual and materialized views, we have several options that relate to our mapping table. We observe that significant costs in querying transformed data lie in (1) pattern-matching and binding queries to nodes and edges (MATCH-WHERE clauses); and (2) the mapping from base-graph nodes and edges to view nodes and edges. Thus, intuitively we should be able to pre-index some of these components.

Input Side			Output Side		
f_1	r	p	f_1	u	p
1	10	2	1	$f_w(1, 10, 2)$	2
3	11	4	3	$f_w(3, 11, 4)$	4

Table 1. An instance of subgraph substitution mapping (t_1) derived from a transformation rule in Example 2.1 over some input graph G . The input side is captured by the ASR relation (t_6), and both sides are by the SSR relation (t_8).

5.1 Storing Relations for Subgraph Matching

Graph views start by finding bindings to subgraphs of the input graph, as matched by a MATCH pattern with any WHERE constraints. In a relational implementation, this matching is join-intensive. Materializing this matched result speeds up query answering. An *access support relation* [35] is an indexing method, first proposed for object-oriented databases, that stores matches to path expressions over graph-structured data, as tuples of bindings to nodes and edges. It generally uses tuple IDs but can equivalently be thought of as encoding tuple keys. A transformation rule r of the form $\phi(\bar{X}) \rightarrow \exists \bar{Y}(\psi(\bar{X}, \bar{Y}))$ can be decomposed into two TGDs. One is for generating an ASR:

$$\phi(\bar{X}) \rightarrow \text{asr}(\bar{X}) \quad (t_6)$$

and the other is for defining transformation over the ASR:

$$\text{asr}(\bar{X}) \rightarrow \exists \bar{Y}(\psi(\bar{X}, \bar{Y})) \quad (t_7)$$

The input side of the mapping in Table 1 illustrates how one entry in the ASR captures the bindings to nodes 1, 2 (resp. 3, 4) and edges 10 (resp. 11), allowing direct lookup of the bindings to that subgraph while answering the query unfolded with the view's transformation rules. When answering queries using ASRs, inverse rules are still employed, but in conjunction with the TGD that defines transformation over the ASRs (t_7).

5.2 Subgraph Substitution Relations (SSRs)

An ASR provides a useful way of storing bindings to input subgraphs for a given transformation rule — but it is lacking in information about the *output* subgraph produced in response which the user's query is posed over. To address this gap, we introduce the notion of the *subgraph substitution relation* (SSR). An SSR encodes local transformations made by a transformation rule on an input graph. The local transformations can be formalized as *subgraph substitution mapping* either (1) directly translated to the TGD (e.g., (t_1)) of a construction rule (with CONSTRUCT) in Section 2 or (2) derived from by applying the standard chase algorithm [6] to the TGD of a transformation rule (with DELETE) with the default rule in Section 3.

The conjunction of the atoms over the input graph (i.e., N and E) in the LHS is called *input pattern*, and that over the output graph (i.e., N' and E') in the RHS is called *output pattern*. The table in Table 1 represents the mapping of bindings to nodes and edges from input to output subgraphs by local transformations.

Creating SSRs. An SSR contains a set of subgraphs of the output graph G' . More specifically, it includes all portions originated from input subgraphs and affected (or *transformed*) by graph transformations. For the subgraph substitution mapping of the form $\phi(\bar{X}, \bar{Y}) \rightarrow \exists \bar{Z}(\psi(\bar{X}, \bar{Z}))$, the TGD for generating an SSR is of the form:

$$\phi(\bar{X}, \bar{Y}) \rightarrow \exists \bar{Z}(\text{ssr}(\bar{X}, \bar{Y}, \bar{Z}))$$

EXAMPLE 5.1 (SSR). The TGD (t_1) representing a local transformation from input pattern to output pattern can be decomposed into two TGDs:

$$N(f_1, \text{"Entity"}) \wedge N(p, \text{"Activity"}) \wedge E(r, f_1, p, \text{"read"}) \rightarrow \exists u(\text{ssr}_2(f_1, u, p)), \quad (t_8)$$

$$\text{ssr}_2(f_1, u, p) \rightarrow N'(f_1, \text{"Entity"}) \wedge N'(p, \text{"Activity"}) \wedge E'(u, p, f_1, \text{"Used"}) \wedge EP'(u, \text{"prop"}, 3). \quad (t_9)$$

The TGD (t_8) in the example above shows how an SSR is defined.

Using SSRs in Query Answering. The TGD (t_9) implies that for each tuple in an SSR, there is a subgraph matched by the output pattern in the output graph. The converse is not true in general. If the converse holds, we have the output pattern's complete view over the output graph that can be used in query rewriting. Note we will generally only get a speedup if a query can be rewritten solely in terms of SSRs.

5.3 Query Rewriting with SSRs

We present how we use SSRs in query rewriting as a variation of *optimizing queries using views* [19, 34]. We first populate a set of equivalent rewriting rules from the output pattern and the definition of an SSR derived from each rule, with covered query test defined below. Each rewriting rule is of the form:

$$\psi(\bar{X}) \leftrightarrow \text{ssr}(\bar{X}, \bar{Y})$$

where ψ is a conjunctive query over G' , and $\bar{Y}$ are variables not appearing in ψ . In query rewriting, if a subquery of a query is contained in the LHS of a rewriting rule, it can be substituted by an SSR relation with appropriate attributes.

Given a query Q over a view's output graph G' , the subquery q of Q that matches the body of a valid SSR rewriting rule is replaced by the head of the SSR, using standard techniques from inverse rules [5]. This requires us to determine whether an SSR is sufficient to answer a path-pattern of the query comprehensively.

Whether an SSR contains all matches of some patterns in the output graph G' can be determined by the *covered query test* below with the transformation rule r corresponding to the SSR, and graph schema Σ of the input graph G .

Covered Query Test. Intuitively, a query Q passes the test if its body matches a single node or edge with a new label as they are created by transformations only and stored in the SSR. Otherwise, a query Q passes the test *if and only if* there exists no $G \models \Sigma$ such that some matches of Q in G' are not stored in the SSR.

This is a *query containment problem*: we must test if Q_1 is contained in Q_2 under a set Σ of EGDs that can express integrity constraints of base graph G . In our setting, Q_1 and Q_2 are unions of conjunctive queries with negations (UCQ⁻) in the presence of node or edge deletions. Following the standard query containment testing of UCQ⁻ [7, 40], we construct canonical databases from Q_1 and apply the standard chase algorithm with Σ [6]. Note that this is not a performance bottleneck since the chase algorithm is performed only over a small number of canonical databases in practice and once during the SSR creation.

Queries that can be rewritten solely by SSRs benefit the most, as they eliminate the need to access the base graph. However, if only a portion of the query can use SSRs, and the rest still refers to relations in Σ' , it is unfolded and evaluated. Since TGDs used to generate SSRs encode mappings from input to output patterns, we can use the TGDs and the bindings of these mappings to identify which nodes and edges of the input graph should be copied or reconnected in the output graph.

6 EXPERIMENTAL EVALUATION

To compare the efficacy of our techniques for answering queries over graph views, we develop a diverse workload including a synthetic benchmark graph and four real-world graphs: a provenance graph, social network, academic citation graph, and a knowledge graph. We study the performance of both standard GQL/G-CORE views and transformation views in the following aspects:

- the effectiveness of materializing views and answering queries over these views.
- the overhead of incremental view maintenance when updates, including both insertions and deletions, occur in the base graph.
- the performance improvements achieved by using SSRs in query execution.

Our results also allow us to draw conclusions about the performance of different DBMS types.

6.1 Experimental Setup

Our graph view system was implemented as a Java middleware layer, running on the top of off-the-shelf database systems. We used OpenJDK 11.0.16 and Linux Ubuntu 22.04.1 LTS on Amazon EC2 (m5a.xlarge), with AMD EPYC 7571 CPUs at 2.2GHz and 16GB RAM. View materialization times were measured with a cold cache, and query processing and maintenance times were done with a warm cache. Each experiment was carried out eight times, dropping the first; and the maximum and minimum values were excluded in computing the average.

Underlying Database Systems. As the underlying database systems, we chose LogicBlox 4.41.0 (native-Datalog RDBMS), PostgreSQL 14.5 (SQL DBMS), and Neo4j 4.1.4 Community Edition (property graph DBMS).

- Neo4j (N4) is a widely used graph database system and supports the Cypher graph query language. It has no native support for views. We experimented with several schemes for materializing views, as described in Section 4.1.
- LogicBlox (LB) is a Datalog-native DBMS that directly supports view materialization and incremental view maintenance.
- PostgreSQL (PG) is an open-source relational DBMS supporting SQL. It does not support incremental maintenance of materialized views, except though an experimental extension that currently lacks key functionality, so we developed our own view maintenance solution. While commercial systems like Oracle offer Incremental View Maintenance (IVM) support, we had focused on open-source RDBMSs, expecting that triggers will serve as a representative mechanism for IVM implementations across such DBMSs. As noted, IVM extensions are under development in PostgreSQL [1], and in fact use triggers as their foundations.

LogicBlox does not provide options to explicitly create indexes, and the performance is entirely dependent on the built-in query optimizer. In PostgreSQL, we explicitly create indexes for improved performance; additionally, PostgreSQL's query optimizer is more mature than that of LogicBlox, thanks to years of work by both researchers and industry.

Dataset	V	E	Description
LSQB	3.96M	22.20M	Subgraph Query Benchmark
PROV	2.65M	5.13M	Provenance Graph
OAG	18.62M	22.93M	Open Academic Graph
SOC	713K	405K	Social Network
WORD	823K	472K	Knowledge Graph

Table 2. Graph datasets: description and statistics.

Dataset	View	Type	Operation	Queries
LSQB	V_1 : Subgraphs of posts and comments linking authors	Standard	Match	Q_1
	V_2 : Subgraphs of friends: Three connected individuals from the same country	Standard	Match	Q_2
PROV	V_3 : Substitute subgraphs in event log-based graph into PROV (zoom-in)	Transformation	Substitute subgraphs	$Q_3 \sim Q_6$
OAG	V_4 : Add co-authors and the author-venue relationships	Transformation	Add new edges	$Q_7 \sim Q_{10}$
SOC	V_5 : Add friend recommendation edges	Transformation	Add new edges	$Q_{11} \sim Q_{14}$
WORD	V_6 : Abstract interconnected entities (zoom-out)	Transformation	Collapse nodes	$Q_{15} \sim Q_{18}$

Table 3. Summary of views and queries used for evaluation (<https://github.com/PennGraphDB/pg-view>).

Datasets. We used one benchmark graph from LDBC (Linked Data Benchmark Council) [2] for subgraph matching, and for diversity added four real-world graph datasets. All are directed graphs. The real-world graphs comprise a provenance graph (from Wikipedia edit history [39]), citation graph (from Open Academic Graph [41]), social network (from Twitter [39]), and knowledge graph (from WordNet [3]). We took subsets of the whole graph and preprocessed them as needed. Table 2 shows the sizes of the datasets.

Workloads. Our query-view combinations were based on existing, widely used graph database benchmarks. For standard GQL views, we leveraged LSQB [2] by LDBC to identify pertinent queries as the basis for views and devised our own queries over the views based on the rationale. For additional workload diversity over other (social, provenance, and knowledge graph) datasets, we devised new views and queries, due to the absence of established benchmarks: New view workloads comprise a variety of local transformations, condensing or eliding detailed content. Queries are designed to extract specific portions of locally transformed subgraphs in the output graph. Table 3 provides a brief description of view definitions. Table 4 shows the complexity of queries in terms of the number of nodes and edges in queries, as well as the number of SSRs joined when the queries are answered by SSRs. Due to space limits, we share specific view and query definitions in an online benchmark document¹.

Queries	Node variables	Edge steps/variables	SSRs needed
Q_{15}	1	0	1
$Q_4, Q_7, Q_9, Q_{11}, Q_{17}$	2	1	1
Q_{16}	3	2	1
Q_5, Q_{12}, Q_{13}	3	2	2
Q_3, Q_8	3	3	1
Q_{18}	4	3	2
Q_6, Q_{10}	4	4	2
Q_{14}	5	4	4

Table 4. View and query complexity, in terms of path components in queries, and the number of SSRs joined to answer queries (when SSRs are used.)

¹<https://github.com/PennGraphDB/pg-view>

View	Impl.	Materialization		Query execution		
		MV	SSR	$Q_1(G)$	$Q_1(MV)$	$Q_1(SSR)$
LSQB V_1	LB	43.144	43.048	2.341	0.141	0.106
	PG	14.639	6.975	3.196	0.485	0.437
	N4-UP	1.614	-	0.295	0.075	-
	N4-OV	1.120	-	0.223	0.032	-
		MV	SSR	$Q_2(G)$	$Q_2(MV)$	$Q_2(SSR)$
LSQB V_2	LB	54.514	53.675	0.560	0.301	0.160
	PG	47.740	9.414	7.918	4.067	0.431
	N4-UP	6.723	-	0.771	0.516	-
	N4-OV	2.168	-	0.614	11.524	-

Table 5. Execution times of the LSQB workload with GQL/G-CORE views, including materializing the views (MV); creating SSRs; and query execution times over the base graph G , MV, and using SSRs. Our Neo4J implementation does not support SSRs.

Methodology. We separately measured the view vs. query evaluation times, for different storage schemes and datasets. In Neo4j, we implemented two variants of materialized views (detailed in Section 4.1): update in-place (UP) and overlay (OV), due to lack of native view support. For relational storage, we considered materialized views (MV) and subgraph substitution relations (SSRs). To explore how well different schemes support answering queries over views, we measured the query execution times over the views, including virtual views (VV).

6.2 Implementing Standard GQL Views

We first examine the performance of standard views in GQL/G-CORE: the cost of querying over views, the cost of incremental view maintenance, and the effectiveness of SSRs in answering queries. We start with the LSQB benchmark, developed concurrently with the GQL standard. Since the benchmark was developed without views, we adapt a subset of its query workload to separate tasks into views and queries over these views. View V_1 matches 5 nodes and 4 edges; V_2 matches 8 nodes and 7 edges. Queries are posed over (and can solely be answered from) the views.

6.2.1 Answering Queries over Views. View creation and query evaluation times across strategies are shown in Table 5.

General Querying Performance Across Platforms. As a baseline, we consider each platform's query execution times over the raw graph G ($Q(G)$ column). Here, both Neo4J implementations (N4-UP, update-in-place and -OV, overlay) excel, because they are tuned for pattern queries over graphs. On LogicBlox (LB) and PostgreSQL (PG), queries over the raw graph require effective optimization of multiple joins, and due to bad estimates can be 15 times slower.

View Materialization. We next consider the impact of materialized views, starting with the computation overhead (MV column). For both views, LogicBlox (LB) takes longer to materialize results than PostgreSQL (PG) and both Neo4J strategies: it is slowest at processing the joins used in the MATCH. PostgreSQL is up to 3 times faster than LogicBlox. Neo4J's OV strategy performs better than UP because updating properties of selected nodes and edges (OV) is faster than creating (or duplicating) them as new (UP).

As anticipated, the creation of V_2 takes more time than V_1 due to the additional joins involved. However, the increase in time is more noticeable in PostgreSQL compared to LogicBlox. This is because this workload involves a high number of many-to-many relations, resulting in a significant

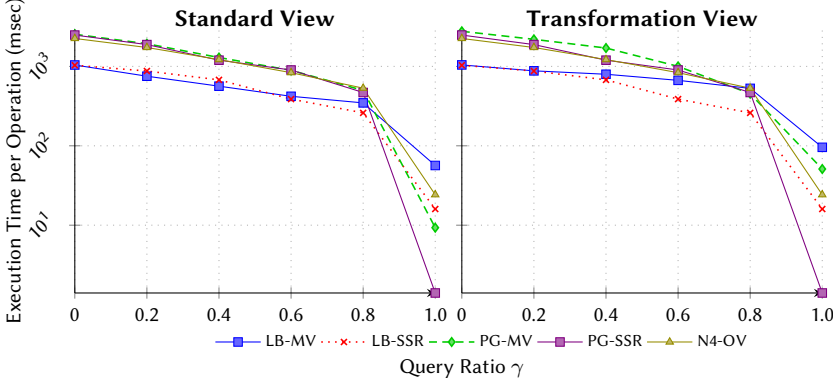


Fig. 3. Mixed workload of insertions and queries on dynamic graphs (LSQB) varying the query ratio. Note that y-axis is on a log scale.

number of duplicated rows being generated during intermediate steps. LogicBlox duplicate-removes earlier than PostgreSQL and thus achieves better scalability.

Querying Views. Comparing the query over the base graph, column $Q(G)$, versus querying over the materialized view, $Q(MV)$, we see significant speedups in the latter case. This is particularly noticeable in LogicBlox, which shows up to 16x improvement, and performs well when handling relatively smaller relations. Even in PostgreSQL, we observed a speedup of up to 6x by leveraging views in answering queries. LogicBlox and PostgreSQL show significant improvements because they avoid many-relation joins. Conversely, the performance in Neo4J declines when querying over views, because with our strategies, view results are materialized within the same database as the raw graph and must be separately filtered out.

Subgraph Substitution Relations (SSRs). SSRs represent a lightweight means of capturing the relationship between bindings from a specific pattern query (TGD) and an output subgraph for the view. Thus, when a query is posed over this view, a rewriting can be obtained by the inverse rule. Column **SSR** in Table 5 demonstrates that materializing SSRs takes slightly less time than view materialization in LogicBlox. SSRs insert tuples representing combinations of nodes and edges in a pattern or output, whereas MV may insert multiple rows into the view's node and edge relations. The performance gap is even more noticeable in PostgreSQL, where we create indexes in each intermediate materialized view while each SSR consists of a single relation, requiring no such extra cost.

We also find that SSRs provide up to 9x speedups in query performance (column $Q(SSR)$) when compared to MV in PostgreSQL. Again, the SSR's compact ability to store an entire subgraph reduces the number of joins required when answering a query over the subgraph contents in the view.

Overall we see noticeable improvements from SSRs. This is particularly evident in answering queries, as they leverage pre-computed query results, avoiding the need for expensive joins.

6.2.2 Incremental View Maintenance. A drawback of view materialization is that the base data may change, requiring updates to any views or indices. Incremental view maintenance (IVM) enables changes on input graphs to be incrementally propagated to views. While LogicBlox supports this feature, standard PostgreSQL and Neo4J do not. Fortunately, the vast majority of updates are insertions; thus, to understand performance we implemented incremental view maintenance for insertions (restricted to a single level of views) on PostgreSQL and Neo4j, using triggers, user defined functions, and the delta rules method [31]. (Performance would be nearly identical for a full

View	Materialization							
	PROV _{V₃}		OAG _{V₄}		SOC _{V₅}		WORD _{V₆}	
Impl.	MV	SSR	MV	SSR	MV	SSR	MV	SSR
LB	32.221	18.709	350.355	124.915	24.869	18.412	10.709	4.202
PG	77.552	11.267	413.047	69.787	60.985	49.881	170.508	159.627
N4-UP	3.462	-	5.929	-	14.935	-	15.941	-
N4-OV	3.741	-	6.520	-	19.421	-	154.716	-

Table 6. Times for creating views with default rules (in sec).

implementation that also supported deletion.) We used LSQB and created a workload of insertions of a node or edge forming a subgraph to be matched by the MATCH clause in the view definition, and selection queries returning a single subgraph match.

The left half of Figure 3 presents the average execution times with mixed workloads. The x -axis represents the query rate γ ($\gamma = 0$ means only updates, $\gamma = 1$ means only queries), and the y -axis represents time in log-scale. We plot the *average execution time* of each query or update in the workload, with different colors representing the system (LB-, PG-, and N4- representing LogicBlox, PostgreSQL, and Neo4J, respectively) and materialization strategy (-MV for materialized views, -SSR for SSRs, and OV for Neo4J Overlay). (We omit N4-UP because it was always slower than N4-OV). The cost of insertions in our workload is significantly more expensive than selection queries, with at least 10x more in LogicBlox where full view maintenance is built in. This trend is similar but more dramatic in our delta rules implementation on other platforms. View materialization and SSRs are both effective for workloads where over roughly 90% of the workload is comprised of queries, after which maintenance starts to substantially penalize performance.

6.3 Transformation Views with the Default Rule

We now evaluate the performance of views which largely preserve the input graph, using the default rule.

6.3.1 Answering Queries over Views. Going beyond the baseline of LSQB and standard GQL, we evaluate the performance of both view creation and query execution over transformation views, where subgraphs may be collapsed or merged, and where default rules are applied. Here, we have a variety of datasets for which local transformations make sense. In LogicBlox and PostgreSQL, non-transformed nodes and edges are copied by the default rule. We contrast this with the performance in Neo4J, where such nodes and edges are not changed. The results are shown in Table 6 and 7.

Materializing Views and SSRs. View materialization times (MV column) are influenced by several factors: the size of the base graph, the complexity of the graph pattern in the view definition, and the proportion of matches within the base graph. Additional computations, such as calculating Skolem functions (to merge nodes/edges) and the cost associated with copying unchanged portions, also play a significant role. For example, only a very small portion of OAG is transformed, but creating MV takes significantly longer than any others due to the size of the base graph. While SOC is a relatively small graph, it requires several self-joins that lead to a high evaluation cost. On the other hand, both PROV and WORD require the creation of multiple new nodes and edges through Skolem functions, which increases the time required for view evaluation. LogicBlox and Postgres show similar trends in their results, and Neo4j consistently takes significantly less time in all cases. This

Query execution												
View = PROV _{V₃}												
Impl.	Q ₃ (VV)	Q ₃ (MV)	Q ₃ (SSR)	Q ₄ (VV)	Q ₄ (MV)	Q ₄ (SSR)	Q ₅ (VV)	Q ₅ (MV)	Q ₅ (SSR)	Q ₆ (VV)	Q ₆ (MV)	Q ₆ (SSR)
LB	t/o	0.169	0.117	t/o	0.116	0.122	t/o	0.114	0.093	t/o	0.153	0.088
PG	83.773	0.431	0.419	29.818	0.418	0.422	55.047	0.435	0.420	t/o	0.465	0.420
N4-UP	-	0.086	-	-	1.227	-	-	1.241	-	-	0.026	-
N4-OV	-	0.057	-	-	2.539	-	-	2.493	-	-	-	0.037
View = OAG _{V₄}												
Impl.	Q ₇ (VV)	Q ₇ (MV)	Q ₇ (SSR)	Q ₈ (VV)	Q ₈ (MV)	Q ₈ (SSR)	Q ₉ (VV)	Q ₉ (MV)	Q ₉ (SSR)	Q ₁₀ (VV)	Q ₁₀ (MV)	Q ₁₀ (SSR)
LB	t/o	0.116	0.111	t/o	0.409	0.110	t/o	0.102	0.094	t/o	0.335	0.109
PG	t/o	0.990	0.613	t/o	0.913	0.621	t/o	0.585	0.593	t/o	0.609	0.590
N4-UP	-	16.996	-	-	19.948	-	-	0.062	-	-	0.069	-
N4-OV	-	31.121	-	-	7.243	-	-	0.100	-	-	0.109	-
View = SOC _{V₅}												
Impl.	Q ₁₁ (VV)	Q ₁₁ (MV)	Q ₁₁ (SSR)	Q ₁₂ (VV)	Q ₁₂ (MV)	Q ₁₂ (SSR)	Q ₁₃ (VV)	Q ₁₃ (MV)	Q ₁₃ (SSR)	Q ₁₄ (VV)	Q ₁₄ (MV)	Q ₁₄ (SSR)
LB	t/o	0.148	0.127	t/o	0.169	0.091	t/o	0.668	0.104	t/o	0.839	0.167
PG	47.583	0.301	0.129	103.547	0.665	0.125	94.494	0.657	0.137	182.078	1.616	0.777
N4-UP	-	0.505	-	-	0.541	-	-	0.849	-	-	5.380	-
N4-OV	-	0.994	-	-	1.041	-	-	1.545	-	-	50.578	-
View = WORD _{V₆}												
Impl.	Q ₁₅ (VV)	Q ₁₅ (MV)	Q ₁₅ (SSR)	Q ₁₆ (VV)	Q ₁₆ (MV)	Q ₁₆ (SSR)	Q ₁₇ (VV)	Q ₁₇ (MV)	Q ₁₇ (SSR)	Q ₁₈ (VV)	Q ₁₈ (MV)	Q ₁₈ (SSR)
LB	t/o	0.097	0.099	t/o	0.203	0.117	t/o	0.205	0.129	t/o	0.461	0.128
PG	3.476	0.171	0.178	37.302	0.349	0.228	25.661	0.315	0.262	56.060	0.432	0.243
N4-UP	-	0.011	-	-	0.480	-	-	0.463	-	-	0.181	-
N4-OV	-	0.102	-	-	4.498	-	-	1.117	-	-	50.997	-

Table 7. Times for query execution times over views with default rules (in sec).

can be attributed to our Neo4J implementation's inherent advantage of not duplicating unchanged nodes and edges.

Finally, in the SSR column we note that materializing SSRs is substantially (up to 6.9x) faster than materializing views, and often comparable to Neo4J. As before, SSRs require less time to build than MV because MV copies unchanged nodes and edges while SSR does not. This is more noticeable when the portion of local transformation in the base graph is smaller.

Querying Views. Once a view has been materialized, querying it (**Q(MV)**) is often substantially faster than querying the raw graph (**Q(VV)**) by unfolding virtual views. Therefore, as for standard GQL queries: if multiple queries are to be made over the views, it makes sense to materialize.

The speedup observed in query execution due to SSRs (**Q(SSR)** vs **Q(MV)** and **Q(VV)**) depend on the ratio of the size of the output graph versus the size of SSRs that encode the output subgraphs. The performance improvement by using SSRs is more noticeable in LogicBlox (up to 6.4x) than in PostgreSQL. This is because PostgreSQL already handles efficiently large graphs, with the help of built in query optimizer with indexes. LogicBlox fully takes advantage of access to smaller relations instead of a large ones. With the help of SSRs, query execution times in both LogicBlox and PostgreSQL become comparable to those in Neo4J. The speedup from SSRs becomes significant when the query involves many joins, in fact analogously to the way a native-graph DBMS often achieves efficiency by retrieving small subgraphs as a composite object. SSRs reduce the number of fewer joins and relations (and relations) required in an RDBMS, also more efficiently filtering candidate rows. For example, Q_{12} , Q_{13} and Q_{14} achieve more than 5x speed up by joining a few SSRs

as opposed to joining multiple node and edge relations, whereas Q_4 and Q_{15} use a small number of nodes and edges in queries, resulting in a negligible improvement in speed by using SSRs, compared to MV.

6.3.2 Incremental View Maintenance. As before, we also consider the question of incremental view maintenance. The key question is: does adding the default rule change our conclusions about the mix? Due to space constraints, we focus on one representative query that highlights our results: we revisit the LSQB dataset in Figure 3, where we add a default rule copying the unmapped nodes and edges. Note that in this setting, MV will now *copy* all nodes and edges in the base graph unless they were already mapped. In contrast, SSRs will simply encode the portions of the graph that were transformed, and during query answering the default rule will subtract these from the base graph to obtain the remaining nodes and edges. Similarly, N4-OV will again only materialize the results of the transformations and will reuse tuples from the base graph.

With the above observations in mind, the average update cost of SSR is slightly smaller than that of MV (1.009x in LogicBlox, 1.12x in PostgreSQL), which is also marginal improvement from standard views (1.007x and 1.02x, resp.). As the query rate increases, the average processing time is dominated by querying times. The speedup in query answering by SSR over MV is up to 5x in LogicBlox and 36x in PostgreSQL ($\gamma = 1$). This is a significant improvement from standard views with 3.5x and 6.6x, respectively. In spite of no significant difference in maintenance cost, SSR shows comparable performance vs. MV as the query rate gets higher.

Overall SSRs are the clear choice for improved performance for certain types of queries: they never do worse than the baseline, and provide significant benefits on the majority of the query workload, especially on PostgreSQL.

6.4 Summary of Experimental Results

In general, our experiments revealed that using the Overlay approach for view materialization provided the best performance in Neo4j. However, relational DBMS storage schemes performed surprisingly well, if full view materialization or especially SSRs were used. LogicBlox provides the overall best performance in queries over SSRs, and also the easiest implementation. PostgreSQL with SSRs provides strong performance, along with better optimization for path queries requiring joins; scale-out implementations and a plethora of extensions.

The key takeaways from our experimental results, for each performance aspect mentioned earlier in this section, are as follows:

- Queries over virtual views can be extremely costly. Materialized Views (MV) often demonstrate superior performance, even with initial materialization cost.
- Incremental View Maintenance (IVM) adds minimal overhead. When compared, SSR performs better than MV in both IVM overhead and query time.
- SSRs can either match or surpass the query performance of fully materialized output views (MVs). Notably, SSRs also benefit from the reduced construction time required compared to MV.

Finally, SSRs enable RDBMSs (in our evaluation, LogicBlox and PostgreSQL) to outperform Neo4j when queries are fully covered by SSRs. This suggests that (with appropriate indexing) relational DBMSes can be a viable and competitive way of not only supporting graph databases, but even graph views.

7 RELATED WORK

The problem of defining, composing [33], maintaining, and indexing views has been studied for many different data models and query languages. Even within the realm of graph and semi-structured data, fundamental differences in the query language and applications have led to fundamentally different results. In the semi-structured view literature, view rewriting [45] and indexing [42] assume paths over a *rooted* graph. Graph query rewriting for SPARQL [37] addresses a model consisting of RDF triples, rather than the more general property graph model that we consider.

Within the property graph space, commercial DBMSs use a wide variety of languages, with Gremlin [47] (a functional graph transformation language) and Cypher/OpenCypher [28] (a true graph query language) as the most popular implementations. GQL/G-CORE [4, 8] seeks to unify their data models and query languages, as well as views or schemas [9, 10]. Research systems like GraphFlowDB [32] and its successor Kùzu [26] develop novel primitives for dedicated graph DBMS storage, query processing, and transaction primitives; Kùzu adopts OpenCypher as its language, and to our knowledge does not support views. Bonifati et al. [15, 16] have investigated numerous problems in answering queries views over graphs from a theoretical perspective. This work heavily informed the GQL specification. However, they are not concerned with graph normalization and abstraction (as proposed, e.g., for provenance graphs [13, 14]), for which we propose “default rules”. Kaskade [20] perhaps relates most closely in spirit to our work, in materializing graph views and rewriting queries to leverage these views. Its query language seems to be a subset of OpenCypher with a grouping clause, which returns nested relation-like output; whereas we support full graph-to-graph transforms as well as subgraph substitution. Finally, we develop RDBMS indexing schemes, in the spirit of the access support relation [35] and the t-index [42].

8 CONCLUSIONS AND FUTURE WORK

In this paper, we investigated semantics strategies for implementing the view capabilities proposed in the emerging property graphs standard, GQL, using existing DBMS platforms. We developed novel strategies for materializing views in property graph DBMSs and for subgraph substitution relations in relational DBMSs. We considered extensions which make it easy to copy all *unmapped* nodes and edges to the view, via the default rule. We demonstrated the performance of our techniques with real-world and synthetic graph datasets under various view and query workloads, and found that RDBMSs with SSRs provided superior overall performance.

In future work, we intend to look at whether the fundamental ideas of SSRs — capturing the bindings of a subgraph, whereas the *structure* of the connections can be specified via a rule — might also be useful for data compression. We also hope to study techniques for sharding or partitioning property graphs.

ACKNOWLEDGEMENTS

The authors would like to thank our anonymous reviewers for their feedback. We also gratefully acknowledge the support of NSF grant IIS-1910108 for portions of this work.

REFERENCES

- [1] [n. d.]. https://github.com/sraoss/pg_ivm.
- [2] [n. d.]. <https://github.com/ldbc/lsqb>.
- [3] [n. d.]. http://www.nltk.org/_modules/nltk/corpus/reader/wordnet.html.
- [4] [n. d.]. <https://www.gqlstandards.org>.
- [5] Serge Abiteboul and Oliver Duschka. 1998. Complexity of answering queries using materialized views. In *PODS*. Seattle, WA, 254–263.

- [6] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of databases*. Vol. 8. Addison-Wesley Reading.
- [7] Foto Afrati and Vassia Pavlaki. 2006. Rewriting queries using views with negation. *AI Communications* 19, 3 (2006), 229–237.
- [8] Renzo Angles, Marcelo Arenas, Pablo Barceló, Peter Boncz, George Fletcher, Claudio Gutierrez, Tobias Lindaaker, Marcus Paradies, Stefan Plantikow, Juan Sequeda, et al. 2018. G-CORE: A core for future graph query languages. In *SIGMOD*. ACM, 1421–1432.
- [9] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Alastair Green, Jan Hidders, Bei Li, Leonid Libkin, Victor Marsault, Wim Martens, et al. 2023. PG-Schema: Schemas for property graphs. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.
- [10] Renzo Angles, Angela Bonifati, Stefania Dumbrava, George Fletcher, Keith W Hare, Jan Hidders, Victor E Lee, Bei Li, Leonid Libkin, Wim Martens, et al. 2021. Pg-keys: Keys for property graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 2423–2436.
- [11] Catriel Beeri and Moshe Y. Vardi. 1984. Formal systems for tuple and equality generating dependencies. *SIAM J. Comput.* 13, 1 (1984), 76–98.
- [12] Olivier Biton, Sarah Cohen Boulakia, Susan B. Davidson, and Carmem S. Hara. 2008. Querying and Managing Provenance through User Views in Scientific Workflows. In *ICDE*. 1072–1081.
- [13] Olivier Biton, Sarah Cohen-Boulakia, Susan B Davidson, and Carmem S Hara. 2008. Querying and managing provenance through user views in scientific workflows. In *2008 IEEE 24th International Conference on Data Engineering*. IEEE, 1072–1081.
- [14] Olivier Biton, Susan B. Davidson, Sanjeev Khanna, and Sudeepa Roy. 2009. Optimizing user views for workflows. In *ICDT*. 310–323.
- [15] Angela Bonifati, Stefania Dumbrava, Joel Dos Santos, Débora Muchaluat-Saade, Cécile Roisin, Nabil Layaïda, Pierre Senellart, Louis Jachiet, Silviu Maniu, Yann Ramusat, et al. 2018. Graph Queries: From Theory to Practice. *SIGMOD RECORD* 47, 4 (2018), 5–16.
- [16] Angela Bonifati, George Fletcher, Hannes Voigt, and Nikolay Yakovets. 2018. Querying graphs. *Synthesis Lectures on Data Management* 10, 3 (2018), 1–184.
- [17] Peter Buneman, Sanjeev Khanna, Keishi Tajima, and Wang-chiew Tan. 2002. Archiving Scientific Data. *Proceedings of the ACM SIGMOD International Conference on Management of Data* 29 (05 2002). <https://doi.org/10.1145/974750.974752>
- [18] Andrea Cali, Georg Gottlob, and Michael Kifer. 2013. Taming the infinite chase: Query answering under expressive relational constraints. *Journal of Artificial Intelligence Research* 48 (2013), 115–174.
- [19] Surajit Chaudhuri, Ravi Krishnamurthy, Spyros Potamianos, and Kyuseok Shim. 1995. Optimizing queries with materialized views. In *Proceedings of the Eleventh International Conference on Data Engineering*. IEEE, 190–200.
- [20] Joana MF da Trindade, Konstantinos Karanasos, Carlo Curino, Samuel Madden, and Julian Shun. 2020. Kaskade: Graph views for efficient graph analytics. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 193–204.
- [21] Alin Deutsch, Nadime Francis, Alastair Green, Keith Hare, Bei Li, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Wim Martens, Jan Michels, et al. 2022. Graph pattern matching in GQL and SQL/PGQ. In *Proceedings of the 2022 International Conference on Management of Data*. 2246–2258.
- [22] A. Deutsch, L.Popa, and V. Tannen. 2006. Query Reformulation with Constraints. *SIGMOD Record* 35, 1 (2006), 65–73.
- [23] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor E Lee. 2020. Aggregation support for modern graph analytics in TigerGraph. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 377–392.
- [24] Ronald Fagin, Phokion G. Kolaitis, and Lucian Popa. 2003. Data exchange: getting to the core. In *PODS*. 90–101.
- [25] Wenfei Fan and Ping Lu. 2019. Dependencies for Graphs. *ACM Trans. Database Syst.* 44, 2, Article 5 (feb 2019), 40 pages. <https://doi.org/10.1145/3287285>
- [26] Xiyang Feng, Guodong Jin, Ziyi Chen, Chang Liu, and Semih Salihoglu. 2023. KÜZU Graph Database Management System. CIDR.
- [27] Ariel Fuxman, Phokion G. Kolaitis, Renée J. Miller, and Wang-Chiew Tan. 2005. Peer data exchange. In *PODS*. 160–171.
- [28] Alastair Green, Martin Junghanns, Max Kießling, Tobias Lindaaker, Stefan Plantikow, and Petra Selmer. 2018. openCypher: New Directions in Property Graph Querying.. In *EDBT*. 520–523.
- [29] Todd J. Green, Grigoris Karvounarakis, Zachary G. Ives, and Val Tannen. 2007. Update Exchange with Mappings and Provenance. In *VLDB*. Amended version available as Univ. of Pennsylvania report MS-CIS-07-26.
- [30] Ashish Gupta and Inderpal Singh Mumick. 1999. Incremental maintenance of recursive views: A survey. (1999).
- [31] Ashish Gupta and Inderpal Singh Mumick (Eds.). 1999. *Materialized Views: Techniques, Implementations and Applications*. The MIT Press.
- [32] Pranjal Gupta, Amine Mhedhbi, and Semih Salihoglu. [n. d.]. Columnar Storage and List-based Processing for Graph Database Management Systems. ([n. d.]).
- [33] Alon Y Halevy. 2000. Theory of answering queries using views. *ACM SIGMOD Record* 29, 4 (2000), 40–47.

- [34] Alon Y. Halevy. 2001. Answering Queries Using Views: A Survey. *VLDB J.* 10, 4 (2001), 270–294.
- [35] Alfons Kemper and Guido Moerkotte. 1990. Advanced Query Processing in Object Bases Using Access Support Relations. In *VLDB*. San Francisco, CA, USA, 290–301.
- [36] David Koop and Jay Patel. 2017. Dataflow notebooks: encoding and tracking dependencies of cells. In *9th USENIX Workshop on the Theory and Practice of Provenance (TaPP 17)*. USENIX Association.
- [37] Wangchao Le, Songyun Duan, Anastasios Kementsietsidis, Feifei Li, and Min Wang. 2011. Rewriting queries on SPARQL views. In *Proceedings of the 20th international conference on World wide web*. ACM, 655–664.
- [38] Maurizio Lenzerini. 2002. Data Integration: A Theoretical Perspective. In *PODS*.
- [39] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [40] Alon Y Levy and Yehoshua Sagiv. 1993. Queries independent of updates. In *VLDB*, Vol. 93. Citeseer, 171–181.
- [41] Microsoft. [n. d.]. Open Academic Graph. <https://www.openacademic.ai/oag/>
- [42] Tova Milo and Dan Suciu. 1999. Index Structures for Path Expressions. In *ICDT*. 277–295.
- [43] Luc Moreau, Paolo Missier, Khalid Belhajjame, Reza B'Far, James Cheney, Sam Coppens, Stephen Cresswell, Yolanda Gil, Paul Groth, Graham Klyne, et al. [n. d.]. PROV-DM: The PROV Data Model. <https://www.w3.org/TR/prov-dm/>
- [44] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo I. Seltzer. 2006. Provenance-Aware Storage Systems. In *USENIX Annual Technical Conference, General Track*. 43–56.
- [45] Yannis Papakonstantinou and Vasilis Vassalos. 1999. Query rewriting for semistructured data. In *ACM SIGMOD Record*, Vol. 28. ACM, 455–466.
- [46] Christine F. Reilly and Jeffrey F. Naughton. 2009. Transparently Gathering Provenance with Provenance Aware Condor. In *TaPP*.
- [47] Marko A Rodriguez. 2015. The Gremlin graph traversal machine and language (invited talk). In *Proceedings of the 15th Symposium on Database Programming Languages*. ACM, 1–10.
- [48] Manolis Stamatogiannakis, Hasanat Kazmi, Hashim Sharif, Remco Vermeulen, Ashish Gehani, Herbert Bos, and Paul Groth. 2016. Trade-offs in automatic provenance capture. In *International Provenance and Annotation Workshop*. Springer, 29–41.
- [49] Oskar van Rest, Sungpack Hong, Jinha Kim, Xuming Meng, and Hassan Chafi. 2016. PGQL: a property graph query language. In *Proceedings of the Fourth International Workshop on Graph Data Management Experiences and Systems*. 1–6.

Received October 2023; revised January 2024; accepted February 2024